1 Right for the Wrong Reasons: Why Language Models Need Explaining

Train two sentiment classifiers on the same movie reviews. The first is a logistic regression, the most transparent model machine learning has to offer, every weight a readable argument. The second is a gradient-boosted tree ensemble: a hundred boosting rounds of opaque decision trees. On a held-out test set drawn from the same distribution as training, the logistic regression scores **0.950** and the ensemble scores **0.955**. By every convention of applied machine learning, you are done. Pick the one with the higher number and ship it.

Then the world shifts. On a second test set (same task, same vocabulary, same label balance, one statistical quirk removed) the two models score **0.515** and **0.517**. On 600 examples, both numbers are statistically indistinguishable from flipping a coin. Not degraded. Not "needs recalibration." *Chance.*

One of those two models told you this would happen: before deployment, in plain sight, for free. The logistic regression's two largest coefficients were **+10.64** and **−10.62**, attached not to sentiment words but to the genre labels *romance* and *horror*. The strongest genuine sentiment word in its vocabulary, *stunning*, carried +2.62, roughly four times smaller. Two features out of 64 held 37.9% of the model's total weight mass. Anyone who glanced at the weights would have said: this is not a sentiment classifier; this is a genre detector wearing a sentiment classifier's badge. The gradient-boosted model contained exactly the same bug, produced the slightly better leaderboard number, and said nothing at all.

The entire experiment (data, both models, the collapse, the confession) runs in 4.7 seconds on a laptop CPU, in this chapter's notebook. That is the price of admission to the question this book spends thirteen chapters answering: *how do you find out what a model's prediction is actually based on, and when can you trust the answer?*

1.1 What this chapter covers

- A five-second Clever-Hans experiment you can run yourself: two models, identical scores, one confession
- Interpretability, explainability, transparency: drawing the lines between three words the field keeps blurring
- Who asks for explanations (developers, end users, auditors, regulators) and why they are asking different questions
- The accuracy-interpretability trade-off, and why it is half a myth
- What changed with LLMs: from "why this label?" to "is this reasoning real?"
- Regulation in one paragraph (the EU AI Act), with the full treatment deferred
- How this book works: the invasiveness ladder, the notebooks, the specimen

Run it as you read: open `ch01-why-explainability.ipynb` (the Colab badge sits at the top), press Run all, and the whole lab finishes in 4.7 seconds on a laptop CPU, no GPU and no deep-learning stack required. Every number below will be on your screen before you reach it here.

1.2 Right for the wrong reasons

Why start a book about explaining language models with a logistic regression on synthetic data? Because the failure mode that motivates the whole field is easiest to *see* when you plant it yourself, and because the fix, reading the model's reasons, is easiest to demonstrate on a model that has readable reasons. Everything after this chapter is about earning the same visibility for models that do not.

In the early 1900s a horse named Clever Hans toured Germany doing arithmetic. Asked for seven minus three, he tapped his hoof four times. Hans was genuinely clever, just not at math: he was reading the involuntary body language of whoever posed the question, and he stopped tapping when the audience tensed up. Right answers, wrong reasons. The wrongness was invisible until a psychologist thought to test him with questioners who did not know the answer themselves. Modern machine learning reproduces this constantly: models latch onto whatever cue predicts the label most cheaply, whether or not that cue has anything to do with the task (Geirhos et al. 2020). Attribution methods applied to award-winning image classifiers have caught them detecting horses via a photographer's copyright watermark (Lapuschkin et al. 2019). Language models do the same: models for natural language inference were caught leaning on a cheap word-overlap heuristic rather than on meaning, and the paper that caught them gave this chapter's title its phrase (McCoy et al. 2019). The literature calls it *shortcut learning*. This lab plants a shortcut on purpose so you can watch the whole lifecycle: learned in training, invisible in evaluation, fatal in deployment.

The corpus is fully synthetic, built from a six-template grammar. Sentiment is carried by three adjective slots per review, drawing from eight positive and eight negative words, but noisily: each slot agrees with the true label with probability 0.75. That noise matters; it is what makes the shortcut tempting. Meanwhile every review names a genre, and in the training data the genre word co-occurs with the label 95% of the time: *romance* with positive, *horror* with negative. From the chapter notebook:

```
def make_corpus(n, cue_corr, rng, cue_pos="romance",
 ↪  cue_neg="horror"):
```

```
    """n balanced reviews; genre word matches the label with
    ↪  probability cue_corr."""
    texts, labels = [], []
    for i in range(n):
        label = i % 2  # 1 = positive, 0 = negative;
↪   perfectly balanced
        own, other = (POS_WORDS, NEG_WORDS) if label == 1
↪   else (NEG_WORDS, POS_WORDS)
        matched = rng.random() < cue_corr
        genre = (cue_pos if label == 1 else cue_neg) if
↪   matched else (cue_neg if label == 1 else cue_pos)
        slots = {f"s{k}": rng.choice(own if rng.random() <
↪   ALIGN else other) for k in (1, 2, 3)}

↪   texts.append(rng.choice(TEMPLATES).format(genre=genre,
↪   **slots))
        labels.append(label)
    return texts, np.array(labels)
```

The full lab is the fastest one in the book: pure scikit-learn, no GPU, 4.7 seconds end to end on a CPU and well under a minute on free Colab. (Colab may ask to restart the runtime after the pinned installs; say yes and rerun.) Every chapter of this book ships a notebook like this one; the prose quotes only the load-bearing lines.

Three datasets come out of that generator: 2,400 training reviews, 600 in-distribution (“iid”) test reviews from the same world, and 600 *shifted* test reviews in which the genre cue is decorrelated from the label: `cue_corr=0.50`, coin-flip genre. The shifted set is our stand-in for deployment: the day horror fans start posting five-star reviews. By design, the genuine sentiment signal is untouched in the shifted set. Only the shortcut dies.

The realized correlation in the actual training sample is worth printing rather than trusting: the cross-tabulation shows *horror* appearing in 1,150 negative reviews against 59 positive, and *romance* in 1,141 positive against 50 negative, about 95.5% as drawn. Then we train the two protagonists on identical TF-IDF features, again from the notebook:

```
vectorizer = TfidfVectorizer()
X_train = vectorizer.fit_transform(train_texts)
X_iid = vectorizer.transform(iid_texts)
X_shift = vectorizer.transform(shift_texts)
feature_names = vectorizer.get_feature_names_out()

lr = LogisticRegression(max_iter=1000,
 ↪  random_state=SEED).fit(X_train, y_train)
gbt =
 ↪  HistGradientBoostingClassifier(random_state=SEED).fit(X_train.toarray(),
 ↪  y_train)
```

Why these two models? They bracket the transparency spectrum while being equally trivial to train, so any difference in what we can *learn from* them is about transparency, not effort. The vocabulary comes out at 64 features. Both models clear 95% on the iid test. And then the shifted set arrives.

Read the two pairs of bars slowly, because the symmetry is the finding. The logistic regression drops from 0.950 to 0.515, a fall of 0.435. The gradient-boosted ensemble drops from 0.955 to 0.517, a fall of 0.438. Both land within noise of 0.5 on a balanced binary task. Neither model was worse than the other, or better; both learned the same lesson from the same data, which is that checking one genre word is cheaper than weighing three noisy adjectives. The iid test set, the instrument the entire field uses to decide what ships, was structurally incapable of noticing, because it was drawn from the same world that contained the shortcut. Accuracy measured where the shortcut works tells you nothing about what happens where it doesn't.

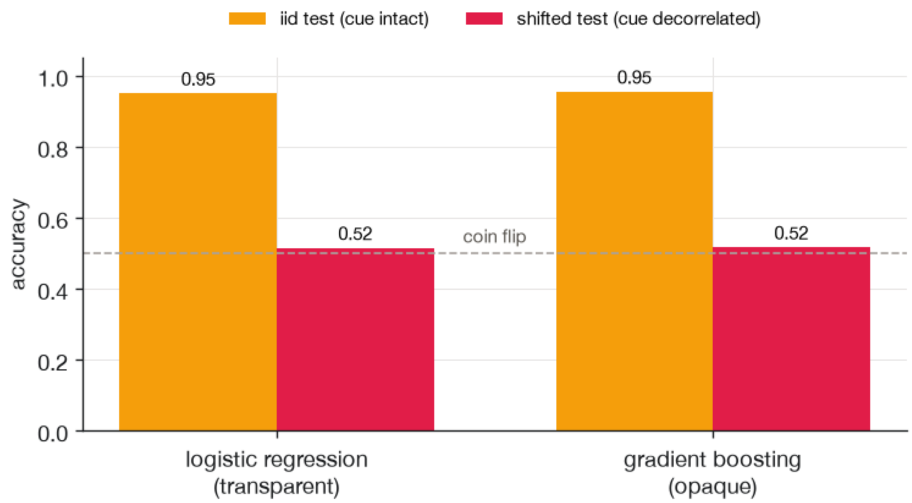

Figure 1.1: Both models collapse from ~95% to a coin flip when the genre cue is decorrelated at test time. The transparent model warned us through its coefficients; the opaque one fails silently.

Sanity check: iid accuracy cannot see shortcut reliance

The most dangerous number in this chapter is 0.955, not 0.517. A held-out test set drawn from the training distribution inherits every spurious correlation the training set had, so a model can score arbitrarily well while being *right for the wrong reasons on every single prediction*. No amount of iid evaluation (more folds, more seeds, bigger test sets) detects this, because the evaluation and the bug share a common cause. The minimum viable defense is the one this lab used: hold out a set where the suspected cue is broken, or read the model's reasons directly, or both. And one non-lesson to avoid taking away: with both models at chance on 600 items, the gap between 0.515 and 0.517 is about one review. Do not conclude that either model is "more robust." Chapter 8 (Chapter 8) turns both of these cautions into measurement protocols.

1.3 The transparent model confessed

The collapse is only half the story. The other half is that one of the two models documented its own bug before deployment, using the cheapest explanation method in this entire book: *read the weights*. A logistic regression over TF-IDF features is an argument you can inspect: each coefficient says how strongly one word pushes the prediction, in which direction, in comparable units. From the chapter notebook:

```
coefs = lr.coef_[0]
order = np.argsort(np.abs(coefs))[::-1][:12]  # top 12 by
↪ |weight|
top_feats = feature_names[order][::-1]
top_coefs = coefs[order][::-1]
genre_words = {"romance", "horror"}
for f, c in zip(top_feats[::-1], top_coefs[::-1]):
    flag = "  <-- genre, not sentiment" if f in genre_words
↪ else ""
    print(f"{f:>12s}  {c:+6.2f}{flag}")
```

The bars in Figure 1.2 do not require statistical sophistication to read; they require only that someone look. *Romance* at +10.64 and *horror* at −10.62 tower over everything else. The strongest genuine sentiment word, *stunning*, manages +2.62; the cue outweighs it roughly fourfold. In log-odds terms the picture is even starker: with the cue at ±10.6 against roughly ±2.3 per adjective slot, the genre word outvotes the sentiment words whenever they are anything short of unanimous. And the auditor's one-line summary, computed in the notebook: two of the model's 64 features hold 37.9% of

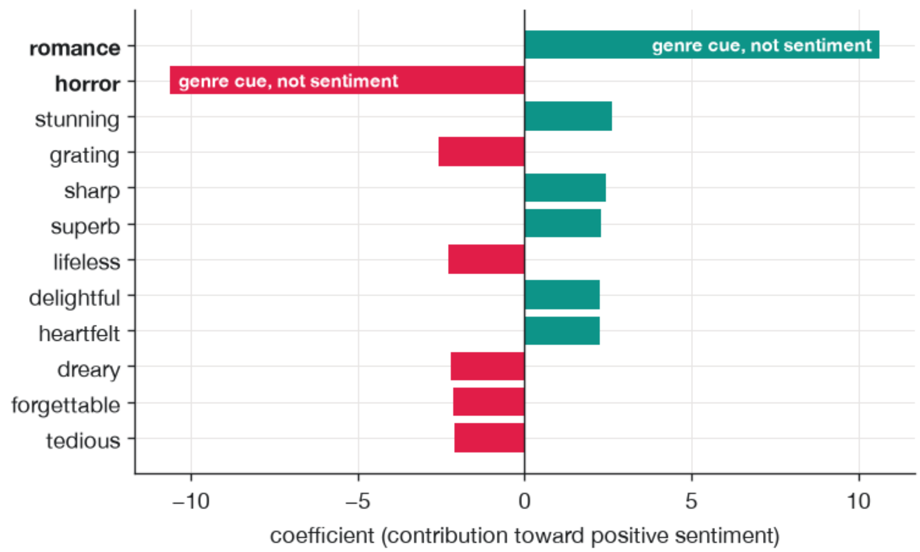

Figure 1.2: The transparent model confesses before deployment: the two genre words dominate the logistic regression's weights, several times larger than any genuine sentiment adjective. Teal pushes toward positive, rose toward negative.

its total absolute weight. If a colleague told you their "sentiment model" spent over a third of its attention on two genre labels, you would not need a deployment failure to know something was wrong.

The gradient-boosted model has no equivalent confession to offer. Its reasoning is distributed across a hundred boosting rounds of trees; there is no `coef_` attribute to print, no single number per word to rank. The same bug is provably inside it (the collapse in Figure 1.1 is the proof), but nothing on the model's surface says so. This asymmetry, not the accuracy gap (there was none worth mentioning), is the real difference between the two models, and it is the difference this book is about. For the logistic regression, the explanation came bundled with the model. For everything more opaque (the gradient-boosted trees here, the 66-million-parameter encoder we adopt in Chapter 4, the frontier LLMs behind APIs), the explanation has to be *extracted*, by methods that cost effort and can themselves be wrong.

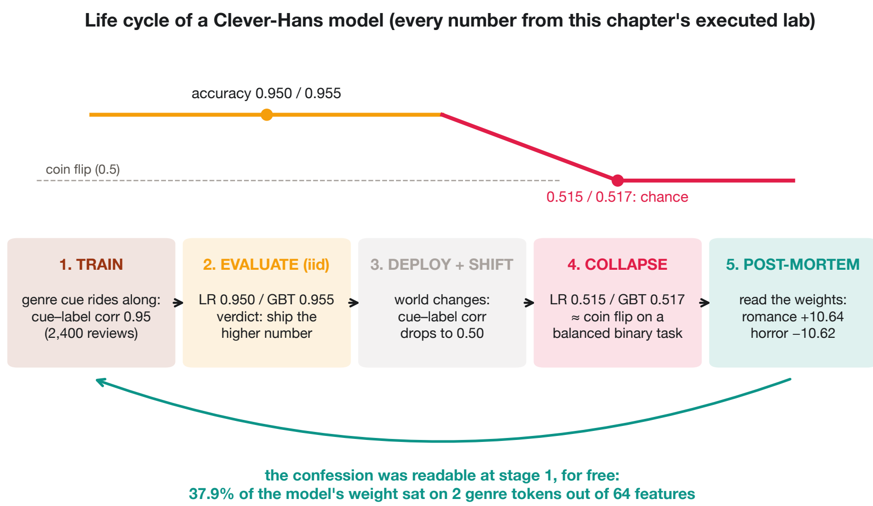

Figure 1.3: The full Clever-Hans lifecycle in one strip. Accuracy falls from 0.950/0.955 to 0.515/0.517 when the cue decorrelates; the post-mortem evidence was readable before anything shipped.

Figure 1.3 compresses the whole lab into the sequence you will see again and again in production post-mortems, with one difference. In real post-mortems, stage 5 happens after the incident review is scheduled. Here it could have happened at stage 1, because the model was transparent enough to interrogate before the leaderboard number was even computed. The rest of this book is about forcing stage 5 leftward for models that resist it.

One honest caveat before moving on: this corpus is deliberately a toy, and the notebook is transparent about it. A six-template grammar with a planted cue is a controlled demonstration of a mechanism, not a claim about movie reviews: iid metrics cannot see shortcut reliance, transparent weights can. The real-world versions of this failure appear on real datasets later in the book: in Chapter 4 the same collapse hits a fine-tuned transformer on real tweets (macro-F1 0.676 on validation falling to 0.357 across a topic shift), and in Chapter 12 a marker planted in real sexism data flips 99.8% of predictions until an attribution penalty pulls its grip down to 12.8%.

1.4 Three words the field keeps blurring

Before this book can be precise about anything else, it has to be precise about its own vocabulary, because "interpretable," "explainable," and "transparent" get used interchangeably in papers, product pages, and regulations, and they name different things (Lipton 2018). Here are the lines as this book draws them.

Transparency is a property of the *model itself*: the degree to which its computation can be directly inspected and followed by a human. The logistic regression above is transparent: 64 weights, one dot product, you can simulate a prediction with a pencil. Transparency comes in degrees (a 10-feature linear model is more transparent than a 10,000-feature one) and it is what you give up as models grow.

Interpretability is a property of the *relationship between a model and a human*: the degree to which a person can understand why the model produced a given output (Doshi-Velez and Kim 2017). A transparent model is interpretable almost by definition. An opaque model can be *made* more interpretable; that is what explanation methods are for.

Explainability, the "X" in XAI, is the *activity and its tooling*: producing artifacts (scores, highlights, counterfactuals, rationales) that account for an opaque model's behavior after the fact. When the model cannot confess on its own, explainability is the interrogation.

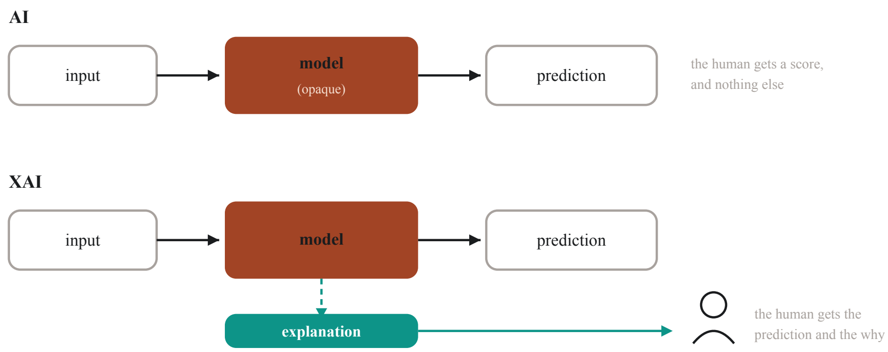

Figure 1.4: A standard AI pipeline and an XAI pipeline, drawn side by side. The only addition is an explanation interface between model and human; the model and its prediction are identical.

Figure 1.4 shows where explainability physically sits: an added interface between the model and the person who has to act on its output. Nothing about the model changes; what changes is what crosses the boundary to the human. That framing also exposes the field's central risk, which deserves naming on page one and gets a full chapter later: an explanation is itself an artifact that can be wrong. The two failure directions have names this book will use relentlessly. An explanation is **faithful** when it tracks the model's real computation. It is **plausible** when it convinces a human. These properties are independent (Jacovi and Goldberg 2020): the coefficient plot in Figure 1.2 is faithful *and* plausible; a fluent paragraph from a chatbot about its own reasoning is plausible and, as Chapter 9 shows experimentally, sometimes silently unfaithful. Whenever you feel an explanation is satisfying, ask which of the two properties produced the feeling.

Figure 1.5 makes the geometry of the risk explicit. Three of the four cells are survivable: the goal cell is where you want to be, the faithful-but-unreadable cell just needs better presentation, and an explanation

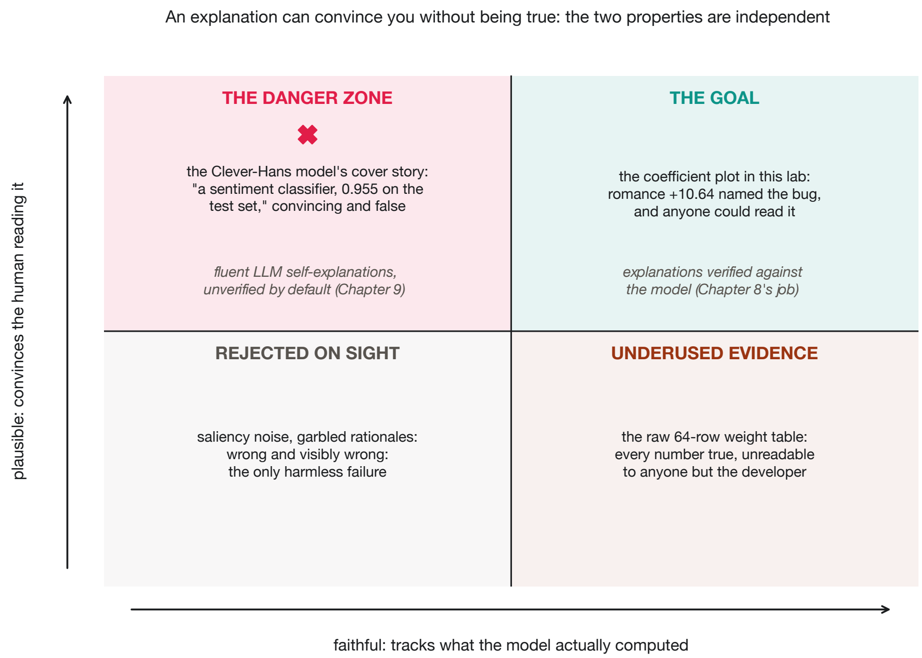

Figure 1.5: The faithfulness-plausibility plane. The top-left cell is the danger zone: the Clever-Hans cover story, 0.955 on the test set, was convincing and false.

that is both wrong and unconvincing gets discarded on sight. Only the plausible-and-unfaithful cell is actively dangerous, because it converts your own judgment into the attack surface. It is the cell that fluent LLM self-explanations occupy by default.

There is also a fourth position worth knowing, argued forcefully by Rudin: for high-stakes decisions, do not explain black boxes after the fact; build transparent models in the first place (Rudin 2019). This chapter's lab is an argument in her favor: the transparent model cost nothing here (0.950 vs. 0.955) and confessed for free. The rest of the book exists because that option is not always on the table, and because with language models it currently never is.

1.5 Who asks, and what they are really asking

"Explain the model" is not one request but at least four, from four audiences, with different questions, different tolerances for technical detail, and different consequences when the answer is bad. Getting this wrong is the most common failure in applied XAI: shipping a developer artifact to an end user, or a comfort artifact to an auditor.

The notebook makes this concrete by rendering the *same* logistic-regression evidence three different ways:

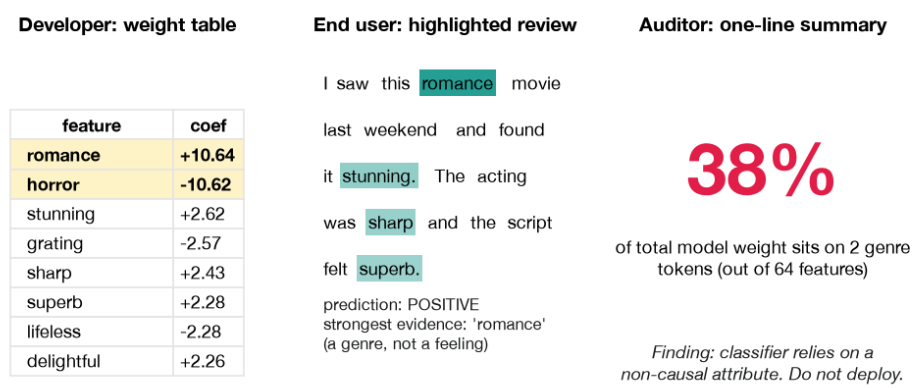

Figure 1.6: The same logistic-regression evidence rendered for three audiences: a coefficient table for the developer, a token-highlighted review for the end user, and a one-number audit statistic for the auditor.

Figure 1.6 shows the same model, the same bug, the same underlying numbers, as three artifacts. The **developer** panel is the raw weight table: dense, quantitative, useless to anyone who does not know what a coefficient is, and exactly what you need to debug the model. The **end-user** panel highlights the words in one specific review (*their* review), showing which tokens pushed the decision; no numbers required, one glance tells the story. The **auditor** panel compresses everything to a single defensible statistic (37.9% of weight mass on two non-sentiment features) plus a verdict, because an auditor is not debugging your model; they are deciding whether to trust your process, and they need a number that survives being quoted in a report.

The fourth audience, missing from the notebook panel because they rarely consume model artifacts directly, is the **regulator**: they ask whether your *system and documentation* meet a legal standard, and their questions arrive as obligations, not API calls. Figure 1.7 adds them and generalizes the pattern.

Use Figure 1.7 as a checklist before building anything: identify the cell first, then choose the method. A developer asking "why did it fail on this input?" wants token-level attributions and error clusters. An end user asking "why was *my* case decided this way?" wants highlighted evidence and, ideally, a counterfactual: what would have changed the outcome. An auditor asking "is this system relying on something it shouldn't?" wants aggregate statistics with stated coverage. A regulator asking "does this meet the transparency obligations?" wants documentation, process, and reproducibility. The deepest technical method in this book cannot rescue an artifact aimed at the wrong cell, and Chapter 2 builds output form into its taxonomy for exactly this reason.

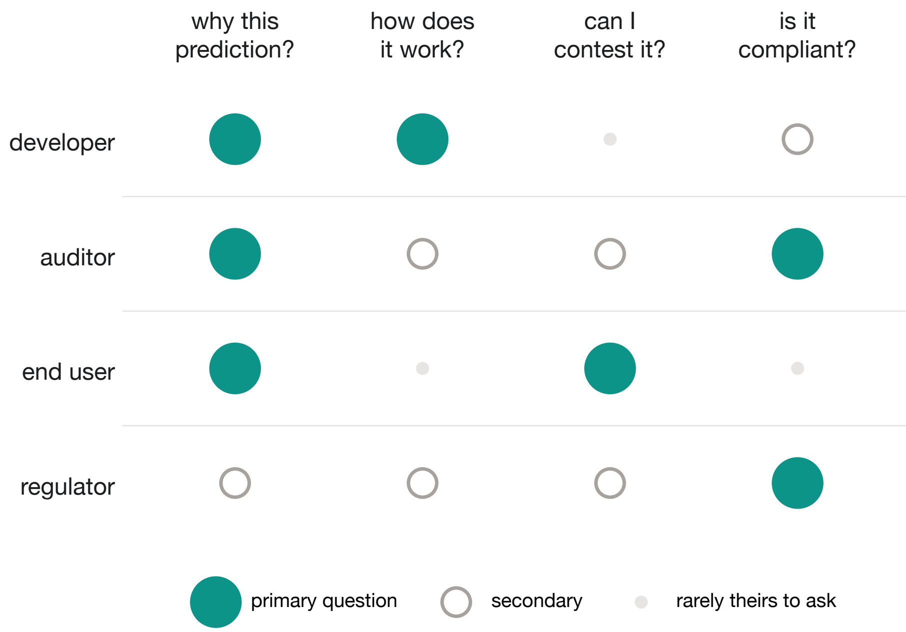

Figure 1.7: The audience-question matrix: four audiences against the four canonical questions, with filled dots marking primary questions. "Why this prediction?" is the only column every audience cares about.

1.6 The trade-off that is half a myth

The most common reason teams give for not using transparent models (or for skipping explainability entirely) is the accuracy-interpretability trade-off: the belief that model quality and model readability sit on a fixed frontier, and every unit of one costs a unit of the other. The belief is half true, and the half that is false causes real damage.

The true half: on unstructured inputs (raw text, images, audio) deep models really do outperform anything a human can read end to end, and

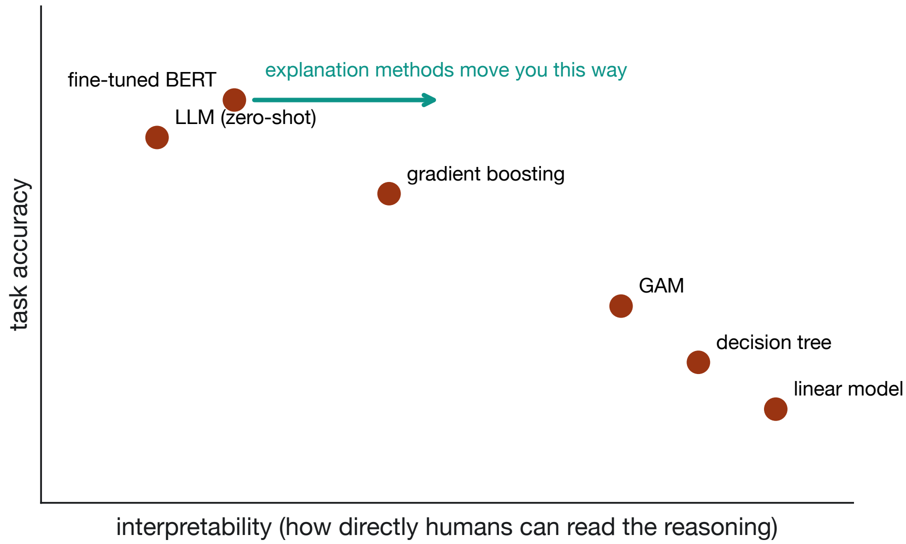

Figure 1.8: The usual accuracy-interpretability frontier, with model families placed schematically. The teal arrow is the point: explanation methods move opaque models toward readability without touching their accuracy.

no linear model recovers what a fine-tuned transformer knows. That gap is real and this book never pretends otherwise; it is why Part II explains transformers rather than replacing them.

The false half is the assumption that *your* problem lives on that frontier. This chapter's lab is a clean counterexample, and the notebook closes the loop with an oracle experiment. If the models collapsed to chance, maybe the shifted task was simply impossible? Honesty demands the check: retrain the same transparent pipeline on a *decorrelated* corpus, where the shortcut never existed, and evaluate on the same shifted test set. From the chapter notebook:

```
clean_texts, y_clean = make_corpus(N_TRAIN, cue_corr=0.50,
 ↪  rng=np.random.default_rng(SEED + 1))
vec_clean = TfidfVectorizer()
lr_clean = LogisticRegression(max_iter=1000,
 ↪  random_state=SEED).fit(
    vec_clean.fit_transform(clean_texts), y_clean
)
oracle_acc = accuracy_score(y_shift,
 ↪  lr_clean.predict(vec_clean.transform(shift_texts)))
```

The result: **0.827** on the shifted set, from the same logistic regression class that just scored 0.515. The sentiment signal was there all along, learnable by the most transparent model available, at accuracy close to the analytic ceiling for this corpus (0.844; see the box below). Nothing about the task forced a trade-off; the shortcut was simply cheaper than the signal, and both models took the discount. When someone invokes the trade-off to justify an unexplained model, the first question to ask is whether anyone actually measured the transparent baseline on *the distribution that matters*. In this lab, the interpretable model was not the accuracy sacrifice but the only model that came with a warning label. Retrained honestly, it was also nearly optimal.

And for the cases where the deep model genuinely is better (most of this book's cases) the arrow in Figure 1.8 is the point: explanation methods exist to move opaque models rightward, buying interpretability without giving back accuracy. The cost shifts from model performance to engineering effort and to a new obligation: verifying that the purchased explanations are faithful (Chapter 8).

The math, if you want it: the 0.844 ceiling

Each review carries three adjective slots, and each slot independently agrees with the true label with probability $p = 0.75$. A classifier that sees only the adjectives can do no better than betting with the majority of the three slots. The probability that at least two of three slots agree with the label is

$$P(\text{majority correct}) = p^3 + 3p^2(1-p) = 0.75^3 + 3(0.75)^2(0.25) = 0.4219 + 0.4219 = 0.844.$$

That is the Bayes ceiling for this corpus once the cue is decorrelated: no model, however deep, can beat 0.844 on the shifted set. The oracle logistic regression reaches 0.827 against it; the remaining gap is within estimation noise for a 600-item evaluation. This is why the chapter can say the collapse to 0.515 was avoidable: 83% of the shifted problem was solvable, and a linear model solves it.

1.7 What changed with LLMs

If shortcut learning were the whole problem, this book could be short: read the weights when you can, run attribution methods when you cannot, hold out a shifted test set, done. Large language models changed the problem statement itself, in two ways that shape everything from Chapter 9 onward.

First, the object being explained changed. A classifier produces a label, and the canonical question is *why this label?*, a question about the influence of inputs on one output. An LLM produces a trajectory: a thousand-token answer, a chain of intermediate reasoning, tool calls, revisions. "Which input token mattered" is still askable and still useful, but it no longer covers the interesting behavior. The unit of explanation shifted from a decision to a *process*.

Second, and this is the new thing, the model became its own explanation interface. Ask a chat model why it answered as it did and it produces a fluent, structured, confident rationale. Hundreds of millions of people now consume these self-explanations daily as if they were the transparent model's coefficient plot. They are not. A stated chain of thought is an output of the very system under audit, optimized to be a good *answer*, not a good *account*; models can be steered by biasing features they never mention, narrating a plausible story while the real computation runs on something else (Turpin et al. 2023). In the vocabulary of this chapter: LLM self-explanations are the first explanation format in history that is maximally plausible by construction and unverified by default. The question stopped being "why this label?" and became **"is this reasoning real?"** Answering it is a measurement problem, not a reading-comprehension problem. Chapter 9 builds the test battery.

The scale shift matters too, less philosophically but just as practically. This chapter's transparent model had 64 features and confessed in one bar chart. The specimen classifier of Part II has 66 million parameters; frontier models have hundreds of billions and arrive behind an API that exposes none of them. Every method in this book is, one way or another, a strategy for recovering some fraction of the confession that the logistic regression gave us for free, at increasing cost, from increasingly hostile terrain.

1.8 The rules are arriving

Explainability stopped being purely an engineering preference in 2024, when the EU AI Act entered into force as the first full-scope AI regulation from a major jurisdiction (European Parliament and Council of the European Union 2024). The Act ties obligations to risk tiers (the higher the tier, the heavier the transparency, oversight, and documentation duties), and language models are explicitly in scope, with additional obligations for general-purpose models. You do not need the legal detail yet, and this book defers it to Chapter 13, where the Act's articles are mapped onto the methods you will by then know how to run. What you need now is the direction of travel: "we could not explain the model" is turning from an engineering confession into a compliance finding, and teams that treat explainability as a bolt-on are discovering that regulators, unlike leaderboards, ask the *why* question first.

1.9 The honest result: nobody wins the robustness contest

Every lab in this book gets a section like this one: the place where the result that did not cooperate gets reported instead of smoothed over. This chapter's version is mild but instructive, and it has three parts.

First, the collapse is *total*, and that surprised us. The corpus still carries a real, learnable 0.83-level sentiment signal in the shifted set (the oracle proves it), yet both models land at 0.515 and 0.517, statistically a coin flip on 600 items. The shortcut did not just dilute the models; it displaced the signal almost entirely. The coefficient plot explains why: with the cue's log-odds around ± 10.6 and each adjective contributing roughly ± 2.3, the genre word outvotes the sentiment slots in every review where they are less than unanimous. The logistic regression is not a sentiment model with a bias; it *is* a genre detector, and the sentiment words function as a tiebreaker that almost never gets to vote.

Second, the temptation this result sets up, and the reason this section exists, is to crown a winner. The gradient-boosted model scored 0.517 against the logistic regression's 0.515 in deployment, and 0.955 against 0.950 in the lab. Both deltas are noise. The 0.002 shifted-set gap is about one review out of 600; the ledger for this chapter is explicit that neither model may be called more robust on this evidence, and the prose you are reading honors that. The instructive part is the iid gap: the *opaque* model won the leaderboard, by half a point, while containing the identical fatal bug. If you take one sentence from this chapter into your next model-selection meeting, take this one: **a leaderboard delta tells you nothing about reasons, and reasons are what fail in deployment.**

Third, a reproducibility quirk worth one honest line. The notebook's closing exercise invites you to plant your own cue pair (the defaults are *friday*/*monday*), and its results are numerically identical to the main run: 0.950 iid, 0.515 shifted. That is not a copy-paste bug: only the cue *strings* change between the two runs, not the random stream that assembles the corpus, so the generated datasets are word-for-word identical up to the swapped cue tokens. Determinism, not coincidence.

None of this is a counsel of despair, and the book would be pointless if it were. The same failure planted here has a training-time cure: Chapter 12 takes a shortcut-broken model from F1 0.000 to 0.665 on the shifted distribution by adding an attribution penalty during training, using explanation methods not to describe the bug but to *fix* it. And the adversarial flip side gets its own chapter too: a shortcut you have not found is a shortcut someone else can exploit, and Chapter 7 attacks the very pipeline this book builds, two word-swaps at a time. Between those two chapters lies the whole arc: find the reasons, verify the reasons, then engineer better ones.

1.10 Do you need XAI, and how much?

Not every system needs a circuit-level autopsy, and pretending otherwise is how explainability gets a reputation as an unaffordable luxury. What every system needs is a *deliberate answer* to the depth question, made before deployment rather than after the incident. Three factors decide it: the stakes of a wrong decision, the audience who will consume the explanation, and the regulatory surface you touch. They map onto four depths:

- **None (monitor only).** No per-decision explanation artifact; you rely on aggregate metrics, drift monitoring, and human review of samples. Legitimate for low-stakes, reversible, internal decisions, with this chapter's caveat that your metrics must include at least one deliberately shifted or stress evaluation, because iid accuracy alone cannot see shortcut reliance.

- **Post-hoc.** Attribution and example-based explanations (Part II of this book) generated on demand, for debugging, error analysis, and answering "why this one?" when someone asks. The default for most production ML.
- **Faithful.** Post-hoc is not enough when the explanation itself carries consequences: appeals, contested decisions, audits. Here the explanations must be *evaluated for faithfulness* (Chapter 8) before anyone relies on them, and the evaluation protocol documented. This is a process commitment, not just a method choice.
- **Mechanistic.** You need to know what the model is *doing*, not just which inputs mattered: safety cases, systematic bias investigations, behaviors that per-input methods cannot localize. Part III's territory (Chapter 10), at Part III's cost.

Stakes of a wrong decision	Primary audience	Regulated (e.g., AI Act high-risk)?	Required depth
Low, reversible (routing, ranking, drafts)	Internal team	No	**None**, but keep a shifted eval
Moderate (triage, prioritization, content flags)	Developers, ops	No	**Post-hoc**
Decisions about people (credit, hiring, moderation with appeals)	End users, auditors	Usually yes	**Faithful** (evaluated + documented)

Stakes of a wrong decision	Primary audience	Regulated (e.g., AI Act high-risk)?	Required depth
Safety-critical, systemic, or the model's behavior is itself under investigation	Safety team, regulators, researchers	Yes, or should be	**Mechanistic** on top of faithful

Two rules make the table and Figure 1.9 usable Monday morning. *Escalate on the maximum, not the average*: if any one factor lands in a lower row (an end-user audience, a regulated domain), the whole system takes that row's depth, regardless of how relaxed the other factors are. And *depth is a floor, not a badge*: running SHAP on a loan-denial model does not make it "faithful-tier"; the evaluation and documentation do. The table tells you what to buy, and Chapter 2's taxonomy and decision tree tell you which specific methods deliver it at the access level you actually have.

1.11 How this book works

This chapter planted a bug and caught it by reading a dozen coefficients off a bar chart. The remaining twelve chapters are that same move, re-executed against progressively less cooperative models. The book has a fixed geography so you always know where you are.

The spine is **the invasiveness ladder**, introduced properly in Chapter 2: methods ordered by what they demand from the model, from "can call it through an API" up through gradients, weights and activations, and finally the training loop itself. The working discipline the ladder encodes: start at the cheapest rung that can answer your question, and climb only when

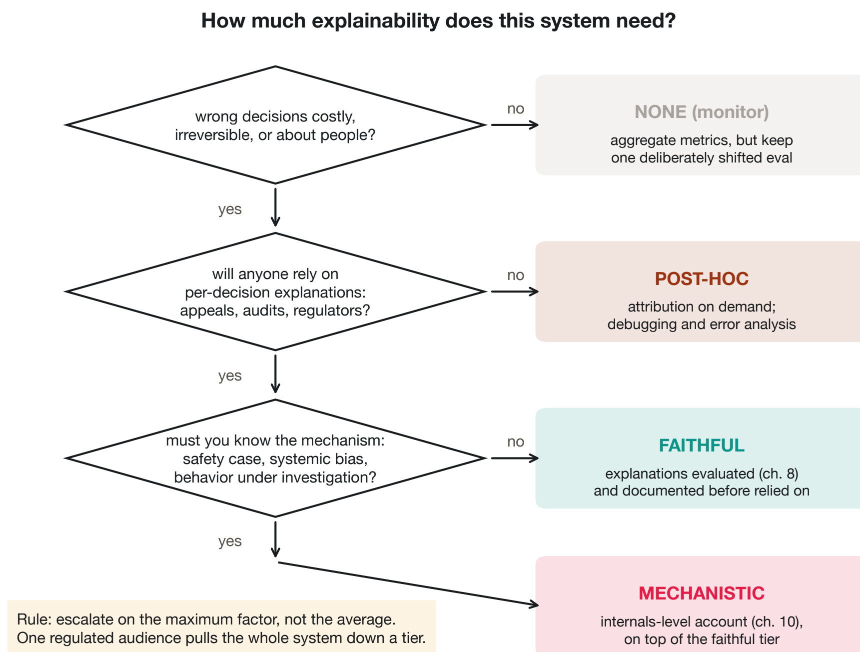

Figure 1.9: Three questions route any system to its required depth: none, post-hoc, faithful, or mechanistic. The amber rule at the bottom left keeps the flowchart honest.

it cannot. This chapter operated on the ladder's ground floor: intrinsic transparency, reading weights the model already exposes. Chapter 2 draws the full map and hands you the method-selection tree the rest of the book navigates by.

The hands-on thread runs through **one executable notebook per chapter**, each opening with a Colab badge, each runnable on free-tier hardware, each with a *smoke mode* (an environment flag that swaps in tiny data) so continuous integration re-executes every lab on a weekly schedule. This chapter's notebook, which needs nothing heavier than scikit-learn, is literally the CI canary. Every number in the prose comes from an executed run, and each chapter's ledger records that run; when a result is ugly, it stays in, in sections titled like the one above.

In Chapter 4 the book adopts **the specimen**: a DistilBERT hate-speech classifier, fine-tuned once and then explained, probed, attacked, and repaired across all of Part II: the shared patient on the table, replacing this chapter's toy. Part II works up the lower rungs on the specimen (attribution, attention, counterfactuals), culminating in the build-and-attack case study of Chapter 7 and the evaluation toolkit of Chapter 8. Part III climbs to LLMs and internals: reasoning under test (Chapter 9), mechanistic tools (Chapter 10), behavioral audits (Chapter 11). Part IV descends back into engineering: training for explainability (Chapter 12, where the TEACH fix lives) and shipping under regulation (Chapter 13).

One ladder, one specimen, one notebook per chapter, no number without a run behind it. That is the contract.

1.12 Summary

- Two models scored 0.950 and 0.955 on a standard test set and 0.515 and 0.517 under deployment shift, statistically chance. **iid accuracy cannot see shortcut reliance**, because the evaluation inherits the training set's spurious correlations.
- The transparent model confessed in advance: ± 10.6 weights on two genre words vs. $+2.62$ for the strongest real sentiment word, 37.9% of weight mass on 2 of 64 features. The opaque model, equally broken, offered no surface to read. That asymmetry, not accuracy, is the difference transparency buys.
- Vocabulary discipline: **transparency** is a property of the model, **interpretability** of the human-model relationship, **explainability** the activity of producing accounts for opaque models. Explanations

are **faithful** when they track the computation and **plausible** when they convince a human: independent properties, never to be blurred.
- Explanations have audiences (developer, end user, auditor, regulator) asking different questions and needing different artifacts; match the cell in Figure 1.7 before choosing a method.
- The accuracy-interpretability trade-off is half a myth: here the transparent oracle hit 0.827 against a 0.844 ceiling. Measure the transparent baseline on the distribution that matters before invoking the trade-off.
- LLMs changed the question from "why this label?" to "is this reasoning real?" Self-explanations are maximally plausible by construction and unverified by default.
- Depth is a decision: none / post-hoc / faithful / mechanistic, set by stakes $\times$ audience $\times$ regulation, escalating on the maximum factor.

1.13 Further reading

The shortcut-learning literature is the deep background for this chapter's lab: Geirhos et al. (2020) synthesizes the phenomenon across vision and NLP and argues it is the default outcome of optimization, not an edge case, while Lapuschkin et al. (2019) is the paper that made "Clever Hans predictor" a term of art by catching classifiers keying on watermarks and copyright tags. On vocabulary and framing, Lipton (2018) remains the essay that forced the field to define interpretability before claiming it, and Doshi-Velez and Kim (2017) supplies the evaluation-first framing this book operationalizes in Chapter 8. Rudin (2019) argues that high-stakes domains should demand inherently transparent models rather than post-hoc accounts, the strongest standing challenge to most of this book's toolkit, and required reading precisely for that reason. Jacovi and Goldberg (2020) draws the faithfulness/plausibility line this book treats as load-bearing, and Turpin et al. (2023) demonstrates why that line matters for LLM self-explanations. For the regulatory landscape previewed here and detailed in Chapter 13, the primary source is the Act itself (European Parliament and Council of the European Union 2024); for a domain-organized survey of the methods this book teaches hands-on, see Mohammadi, Bagheri, et al. (2025).